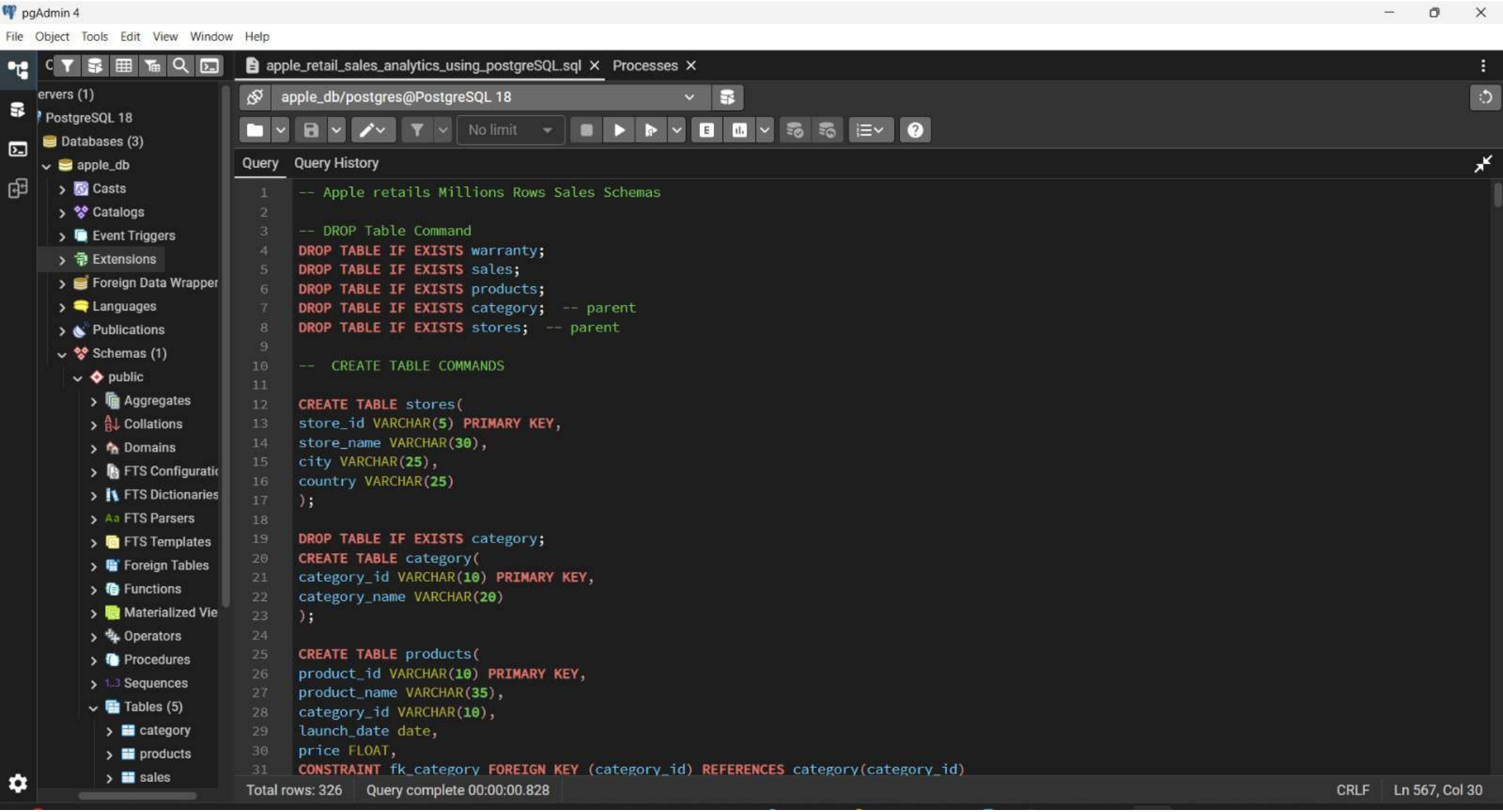
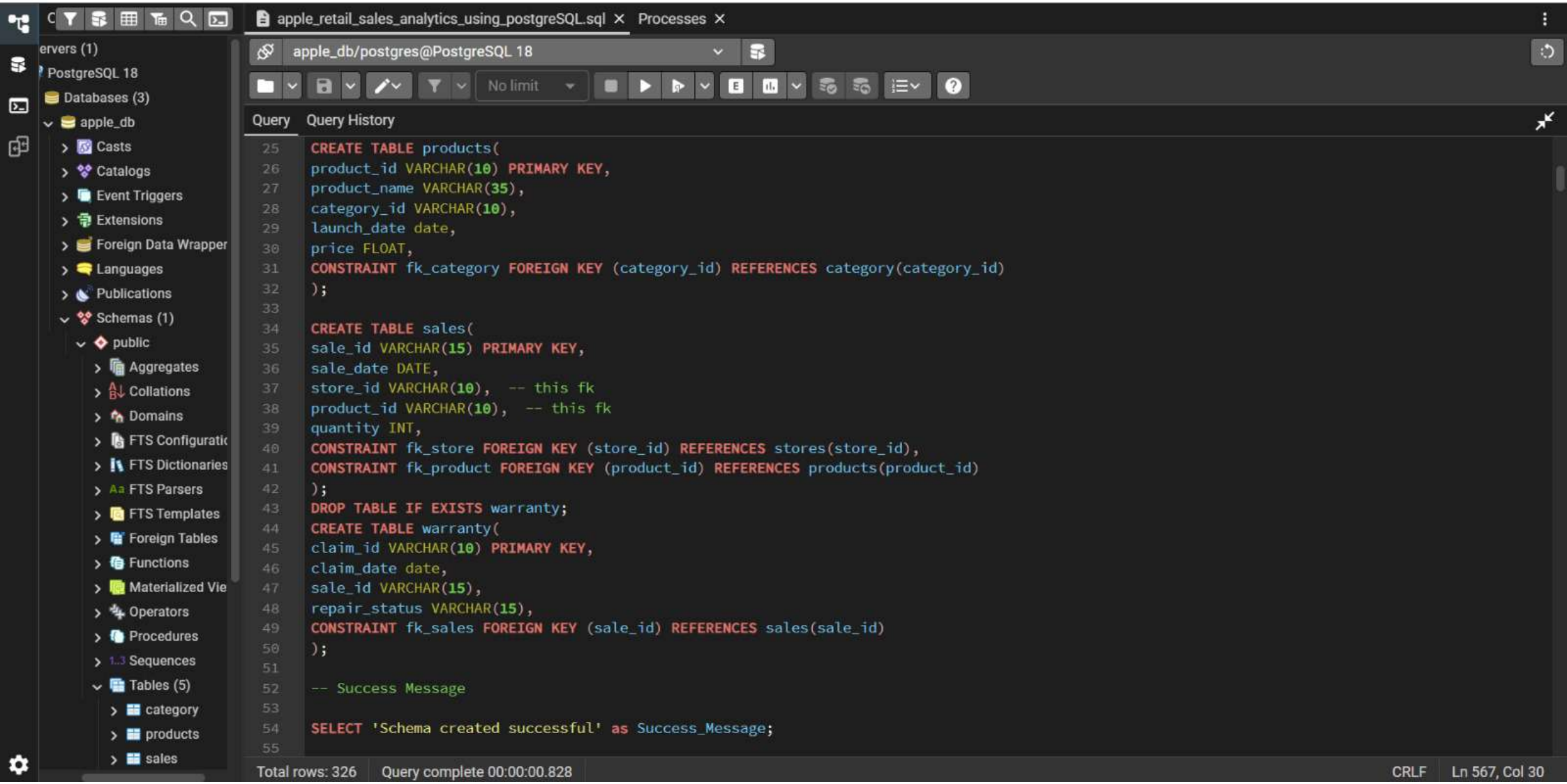
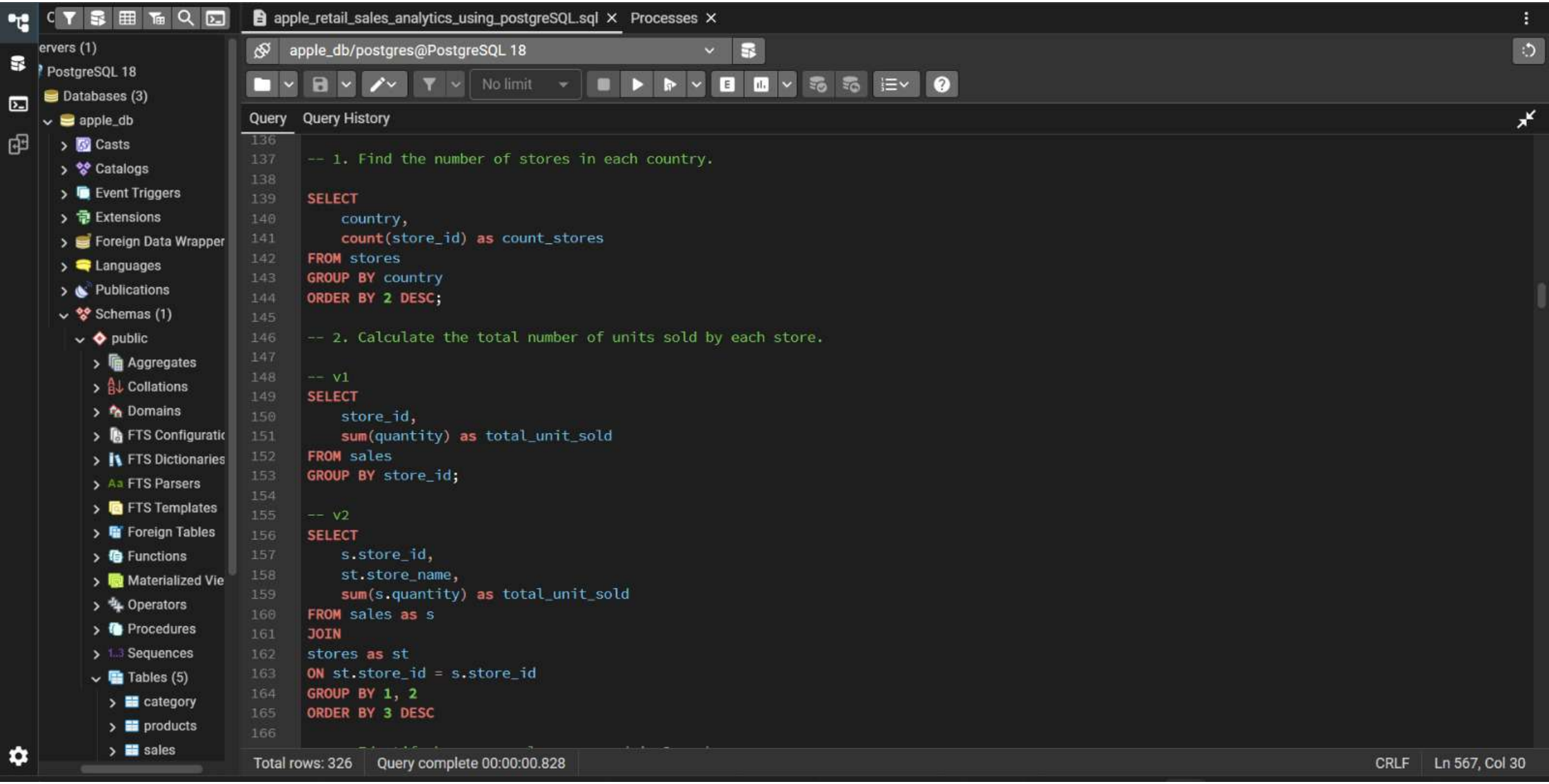
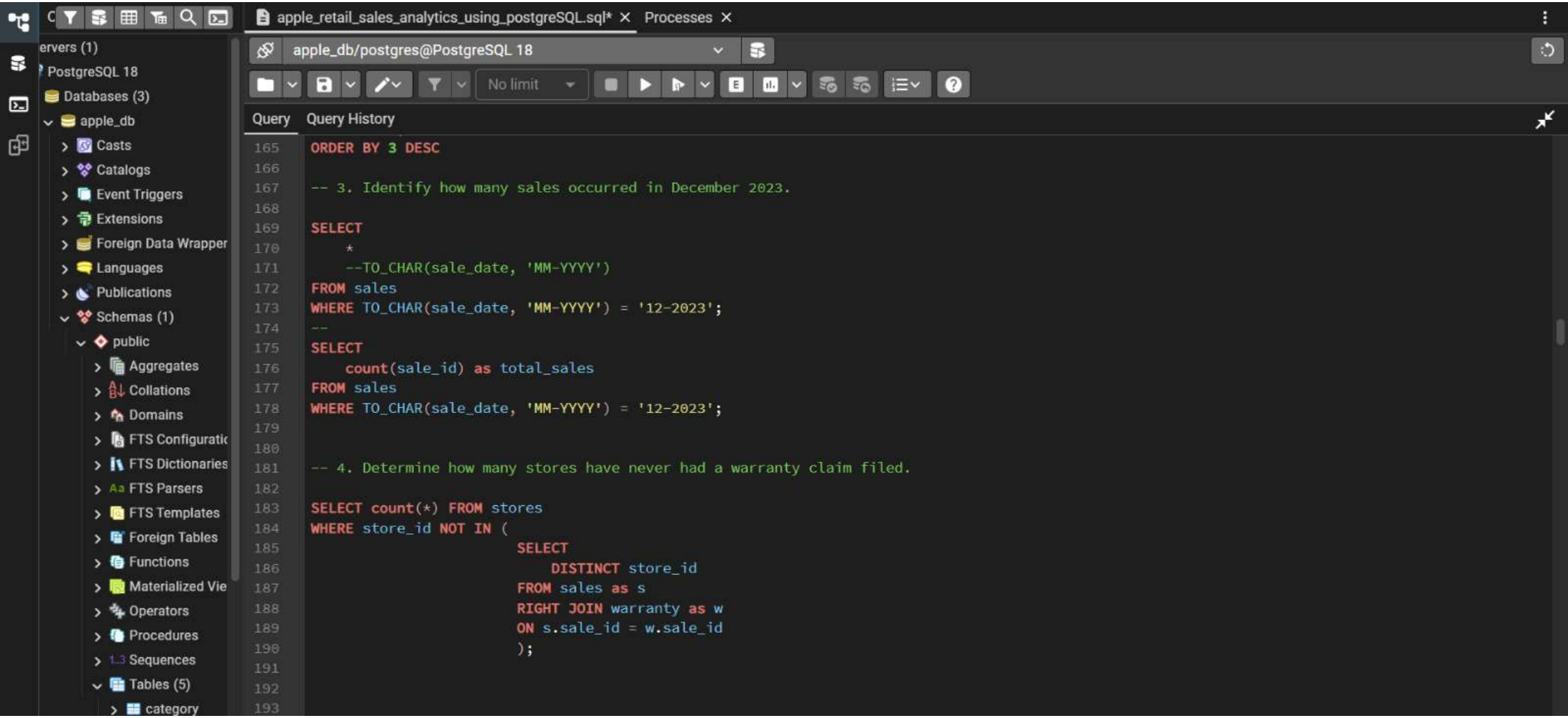
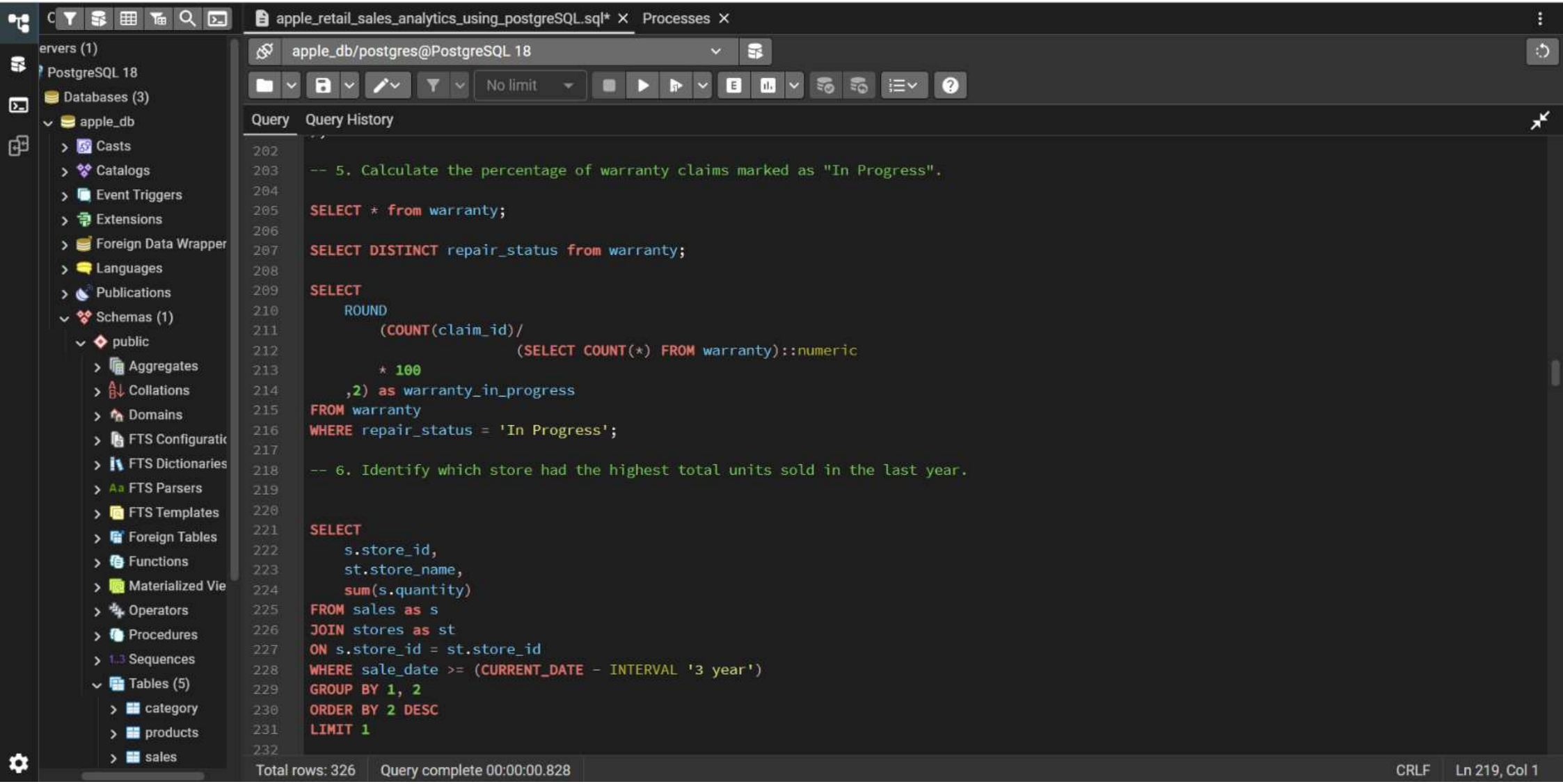
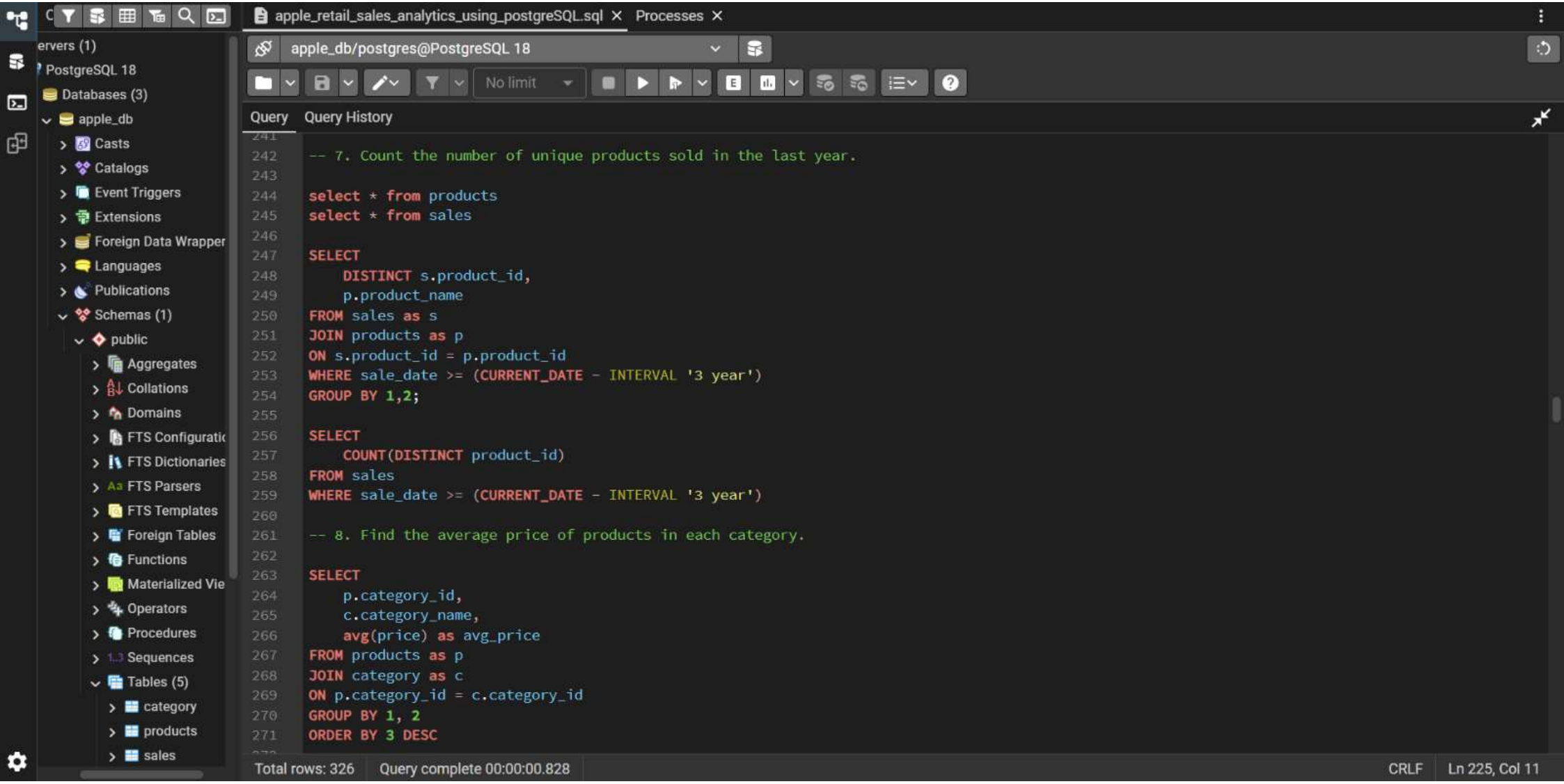

pgAdmin 4
File Object Tools Edit View Window Help
apple_retail_sales_analytics_using_postgreSQL.sql
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
-- Apple retails Millions Rows Sales Schemas
-- DROP Table Command
DROP TABLE IF EXISTS warranty;
DROP TABLE IF EXISTS sales;
DROP TABLE IF EXISTS products;
DROP TABLE IF EXISTS category; -- parent
DROP TABLE IF EXISTS stores; -- parent
-- CREATE TABLE COMMANDS
CREATE TABLE stores(
store_id VARCHAR(5) PRIMARY KEY,
store_name VARCHAR(30),
city VARCHAR(25),
country VARCHAR(25)
);
DROP TABLE IF EXISTS category;
CREATE TABLE category(
category_id VARCHAR(10) PRIMARY KEY,
category_name VARCHAR(20)
);
CREATE TABLE products(
product_id VARCHAR(10) PRIMARY KEY,
product_name VARCHAR(35),
category_id VARCHAR(10),
launch_date date,
price FLOAT,
CONSTRAINT fk_category FOREIGN KEY (category_id) REFERENCES category(category_id)
Servers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratio
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
Sequences
Tables (5)
category
products
sales
Total rows: 326
Query complete 00:00:00.828
CRLF
Ln 567, Col 30

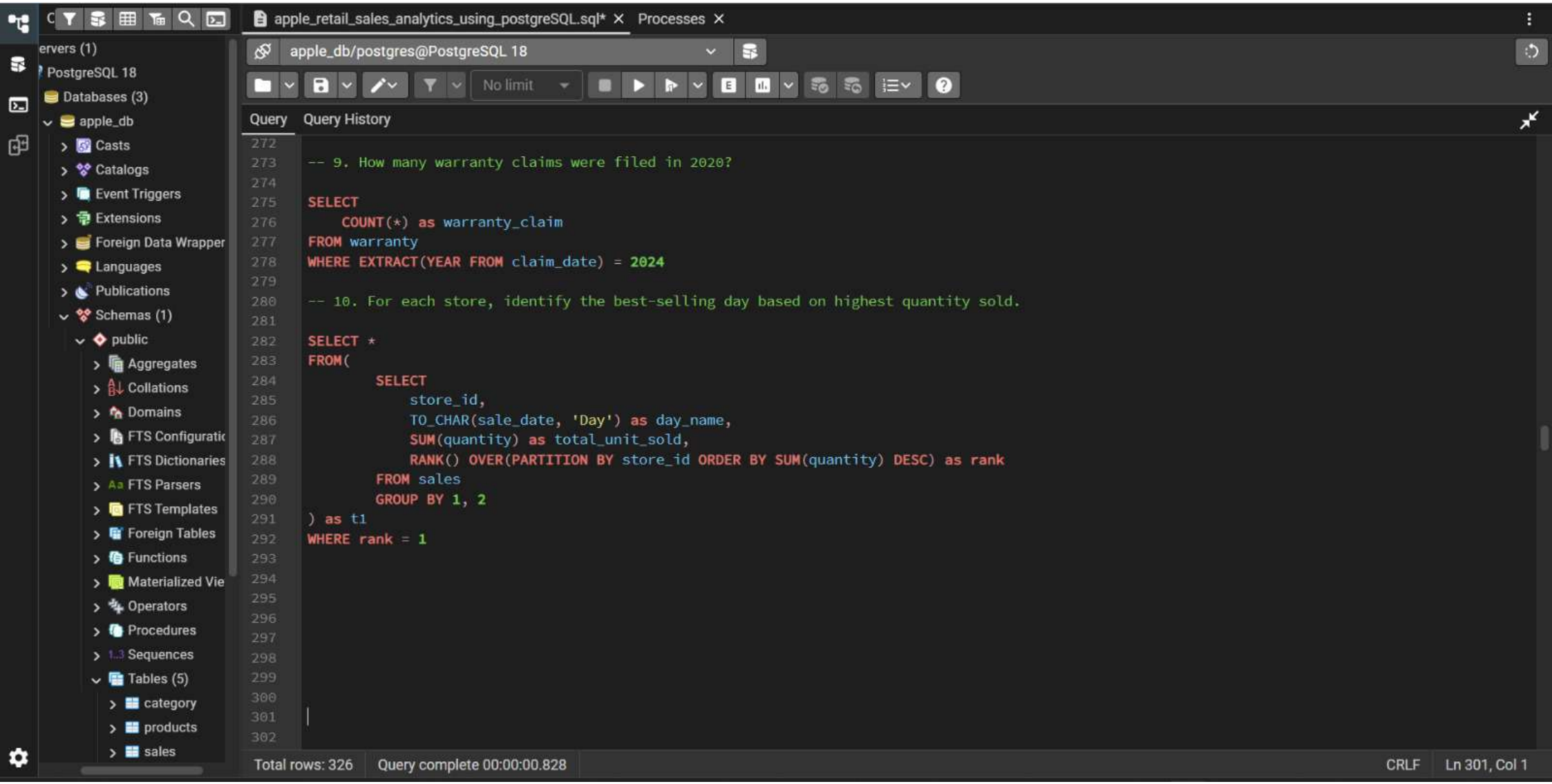

apple_retail_sales_analytics_using_postgreSQL.sql
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
servers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
Sequences
Tables (5)
category
products
sales
25 CREATE TABLE products(
26 product_id VARCHAR(10) PRIMARY KEY,
27 product_name VARCHAR(35),
28 category_id VARCHAR(10),
29 launch_date date,
30 price FLOAT,
31 CONSTRAINT fk_category FOREIGN KEY (category_id) REFERENCES category(category_id)
32 );
33
34 CREATE TABLE sales(
35 sale_id VARCHAR(15) PRIMARY KEY,
36 sale_date DATE,
37 store_id VARCHAR(10), -- this fk
38 product_id VARCHAR(10), -- this fk
39 quantity INT,
40 CONSTRAINT fk_store FOREIGN KEY (store_id) REFERENCES stores(store_id),
41 CONSTRAINT fk_product FOREIGN KEY (product_id) REFERENCES products(product_id)
42 );
43 DROP TABLE IF EXISTS warranty;
44 CREATE TABLE warranty(
45 claim_id VARCHAR(10) PRIMARY KEY,
46 claim_date date,
47 sale_id VARCHAR(15),
48 repair_status VARCHAR(15),
49 CONSTRAINT fk_sales FOREIGN KEY (sale_id) REFERENCES sales(sale_id)
50 );
51
52 -- Success Message
53
54 SELECT 'Schema created successful' as Success_Message;
55
Total rows: 326
Query complete 00:00:00.828
CRLF
Ln 567, Col 30

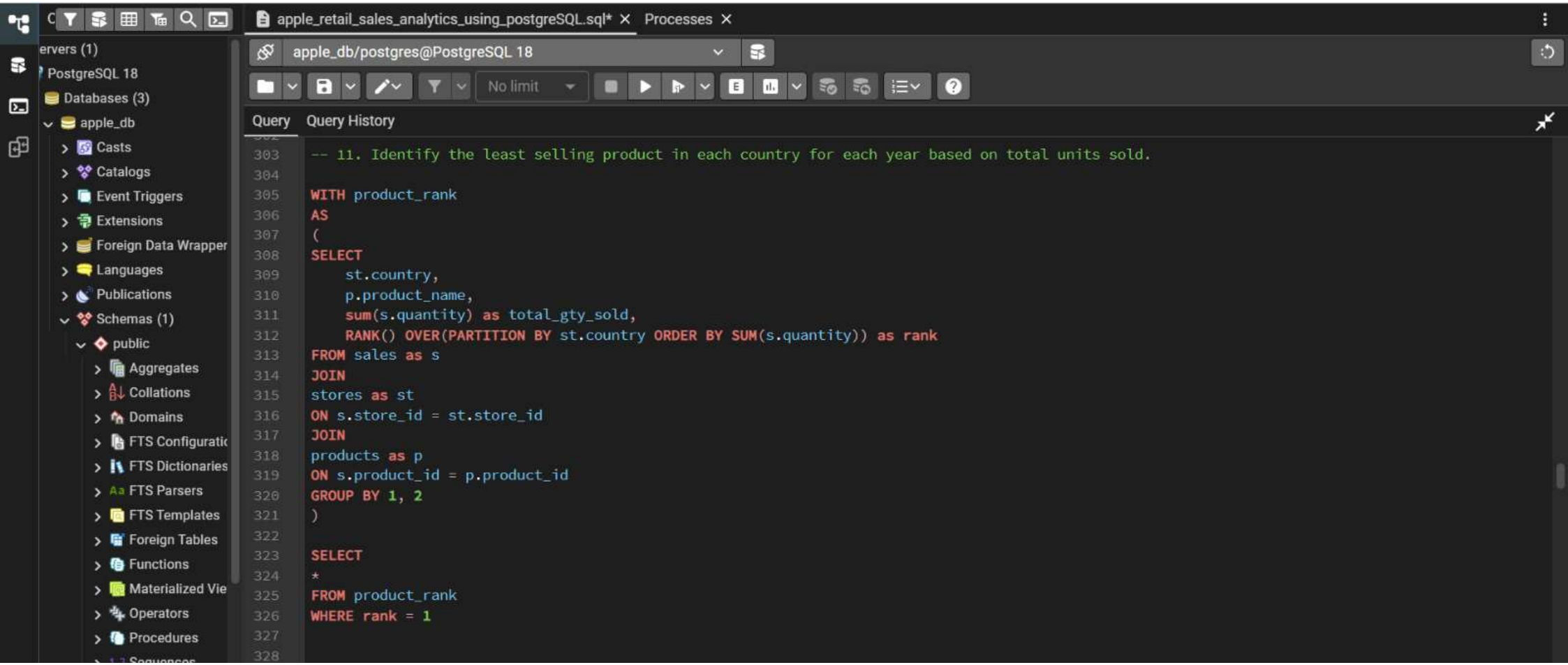

apple_retail_sales_analytics_using_postgreSQL.sql
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
servers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
Sequences
Tables (5)
category
products
sales
136
137 -- 1. Find the number of stores in each country.
138
139 SELECT
140 country,
141 count(store_id) as count_stores
142 FROM stores
143 GROUP BY country
144 ORDER BY 2 DESC;
145
146 -- 2. Calculate the total number of units sold by each store.
147
148 -- v1
149 SELECT
150 store_id,
151 sum(quantity) as total_unit_sold
152 FROM sales
153 GROUP BY store_id;
154
155 -- v2
156 SELECT
157 s.store_id,
158 st.store_name,
159 sum(s.quantity) as total_unit_sold
160 FROM sales as s
161 JOIN
162 stores as st
163 ON st.store_id = s.store_id
164 GROUP BY 1, 2
165 ORDER BY 3 DESC
166
Total rows: 326
Query complete 00:00:00.828
CRLF
Ln 567, Col 30

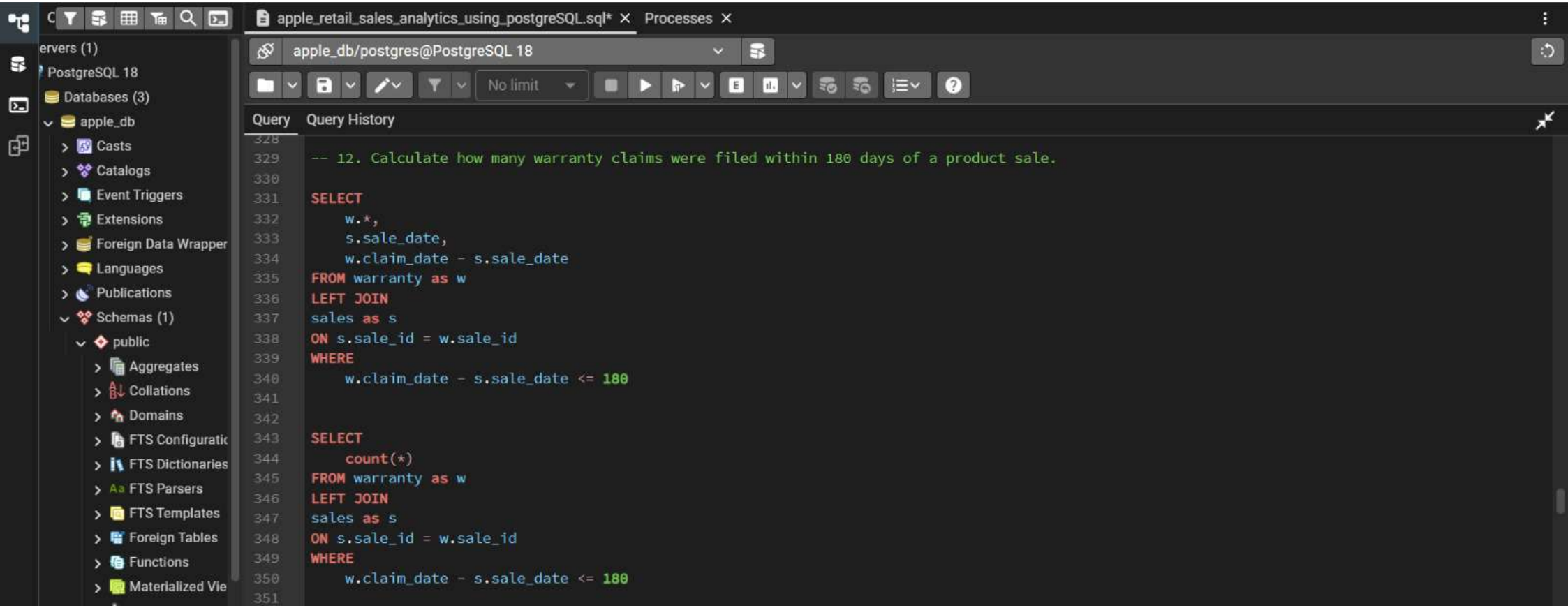

apple_retail_sales_analytics_using_postgreSQL.sql*
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
ervers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
Sequences
Tables (5)
category
ORDER BY 3 DESC
-- 3. Identify how many sales occurred in December 2023.
SELECT
*
--TO_CHAR(sale_date, 'MM-YYYY')
FROM sales
WHERE TO_CHAR(sale_date, 'MM-YYYY') = '12-2023';
--
SELECT
count(sale_id) as total_sales
FROM sales
WHERE TO_CHAR(sale_date, 'MM-YYYY') = '12-2023';
-- 4. Determine how many stores have never had a warranty claim filed.
SELECT count(*) FROM stores
WHERE store_id NOT IN (
SELECT
DISTINCT store_id
FROM sales as s
RIGHT JOIN warranty as w
ON s.sale_id = w.sale_id
);

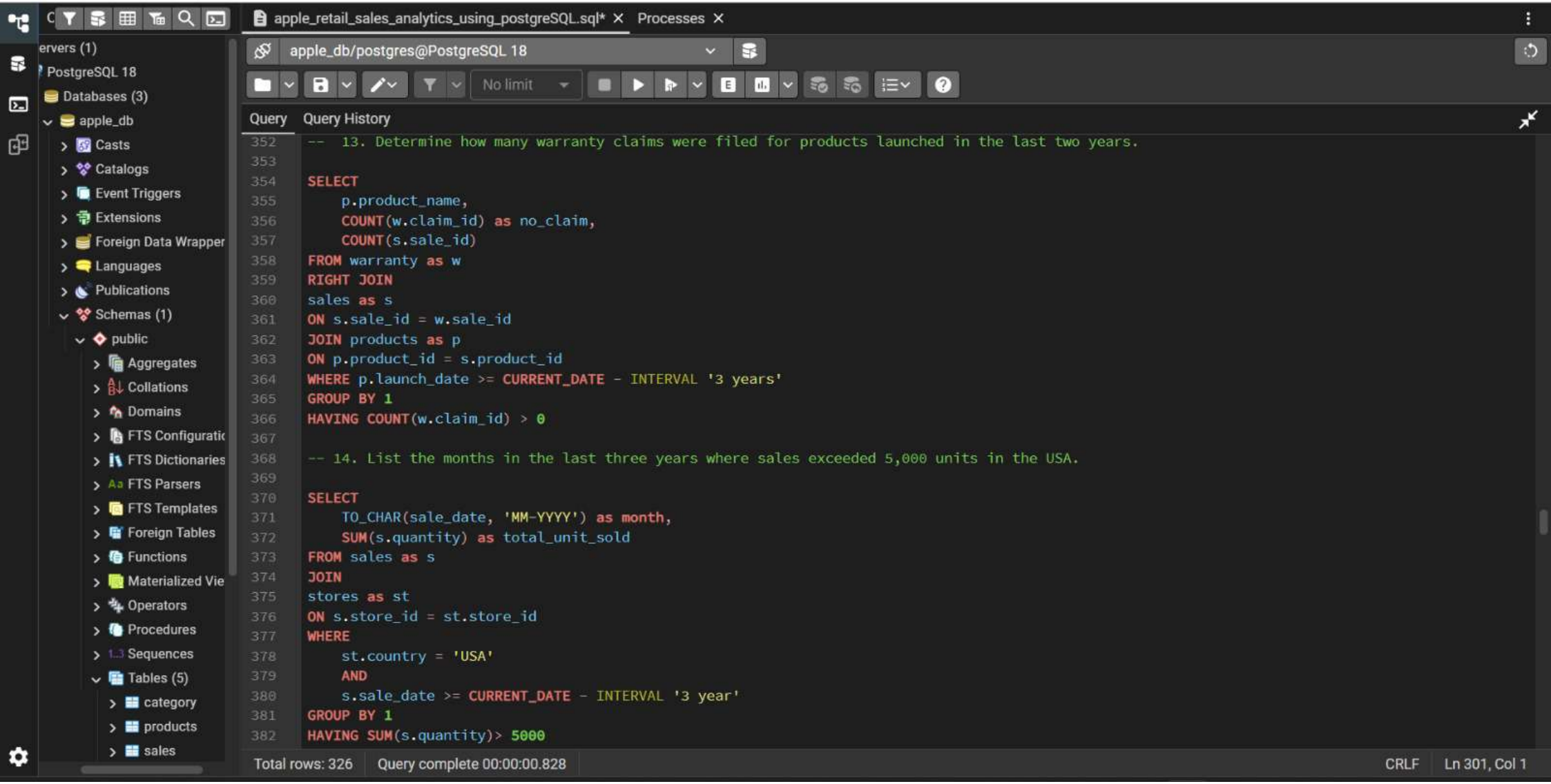

apple_retail_sales_analytics_using_postgreSQL.sql*
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
Servers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
Sequences
Tables (5)
category
products
sales
202
203 -- 5. Calculate the percentage of warranty claims marked as "In Progress".
204
205 SELECT * from warranty;
206
207 SELECT DISTINCT repair_status from warranty;
208
209 SELECT
210 ROUND
211 (COUNT(claim_id)/
212 (SELECT COUNT(*) FROM warranty)::numeric
213 * 100
214 ,2) as warranty_in_progress
215 FROM warranty
216 WHERE repair_status = 'In Progress';
217
218 -- 6. Identify which store had the highest total units sold in the last year.
219
220
221 SELECT
222 s.store_id,
223 st.store_name,
224 sum(s.quantity)
225 FROM sales as s
226 JOIN stores as st
227 ON s.store_id = st.store_id
228 WHERE sale_date >= (CURRENT_DATE - INTERVAL '3 year')
229 GROUP BY 1, 2
230 ORDER BY 2 DESC
231 LIMIT 1
232
Total rows: 326
Query complete 00:00:00.828
CRLF
Ln 219, Col 1

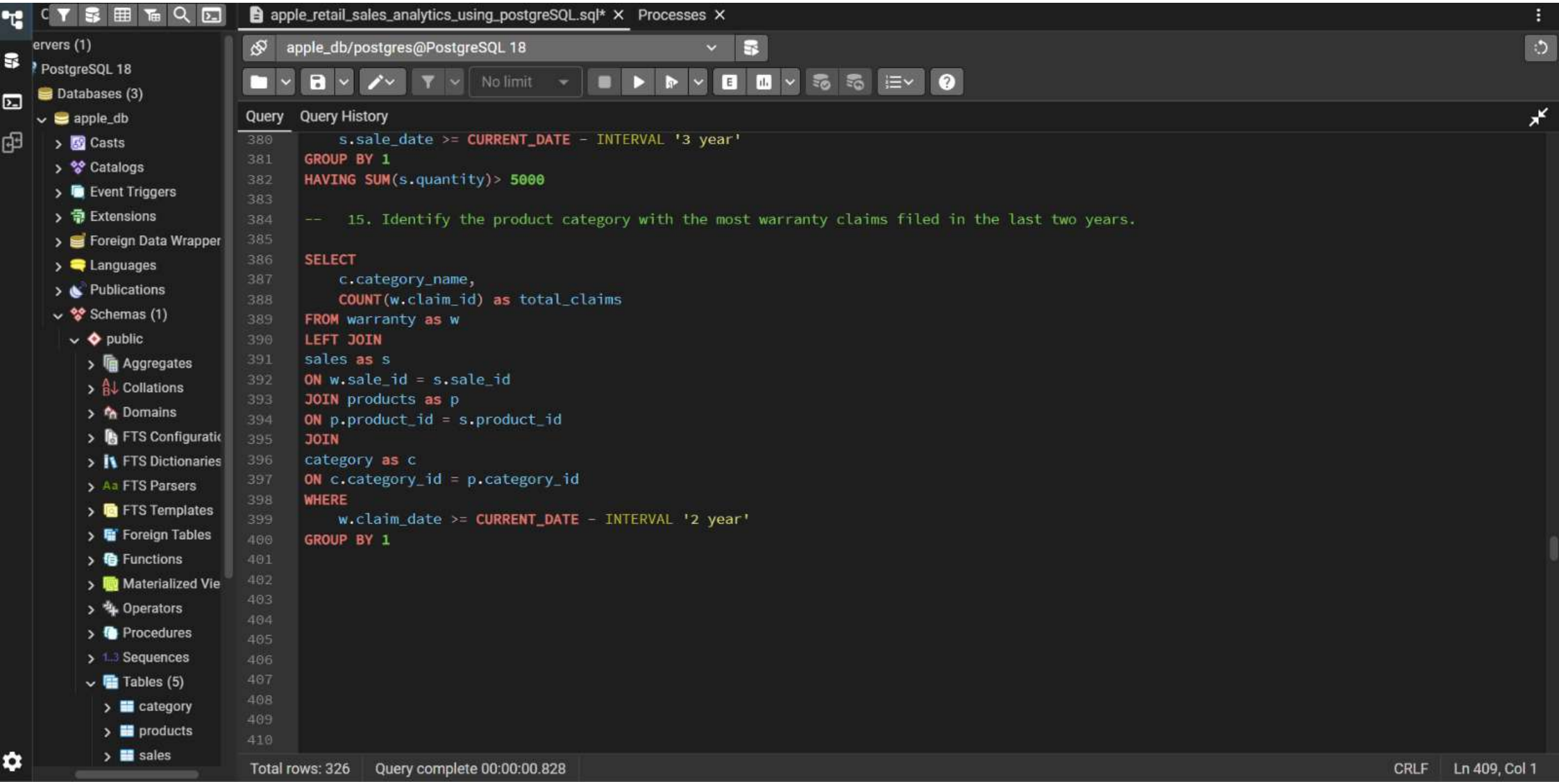

apple_retail_sales_analytics_using_postgreSQL.sql
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
ervers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
Sequences
Tables (5)
category
products
sales
-- 7. Count the number of unique products sold in the last year.
select * from products
select * from sales
SELECT
DISTINCT s.product_id,
p.product_name
FROM sales as s
JOIN products as p
ON s.product_id = p.product_id
WHERE sale_date >= (CURRENT_DATE - INTERVAL '3 year')
GROUP BY 1,2;
SELECT
COUNT(DISTINCT product_id)
FROM sales
WHERE sale_date >= (CURRENT_DATE - INTERVAL '3 year')
-- 8. Find the average price of products in each category.
SELECT
p.category_id,
c.category_name,
avg(price) as avg_price
FROM products as p
JOIN category as c
ON p.category_id = c.category_id
GROUP BY 1, 2
ORDER BY 3 DESC
Total rows: 326
Query complete 00:00:00.828
CRLF
Ln 225, Col 11

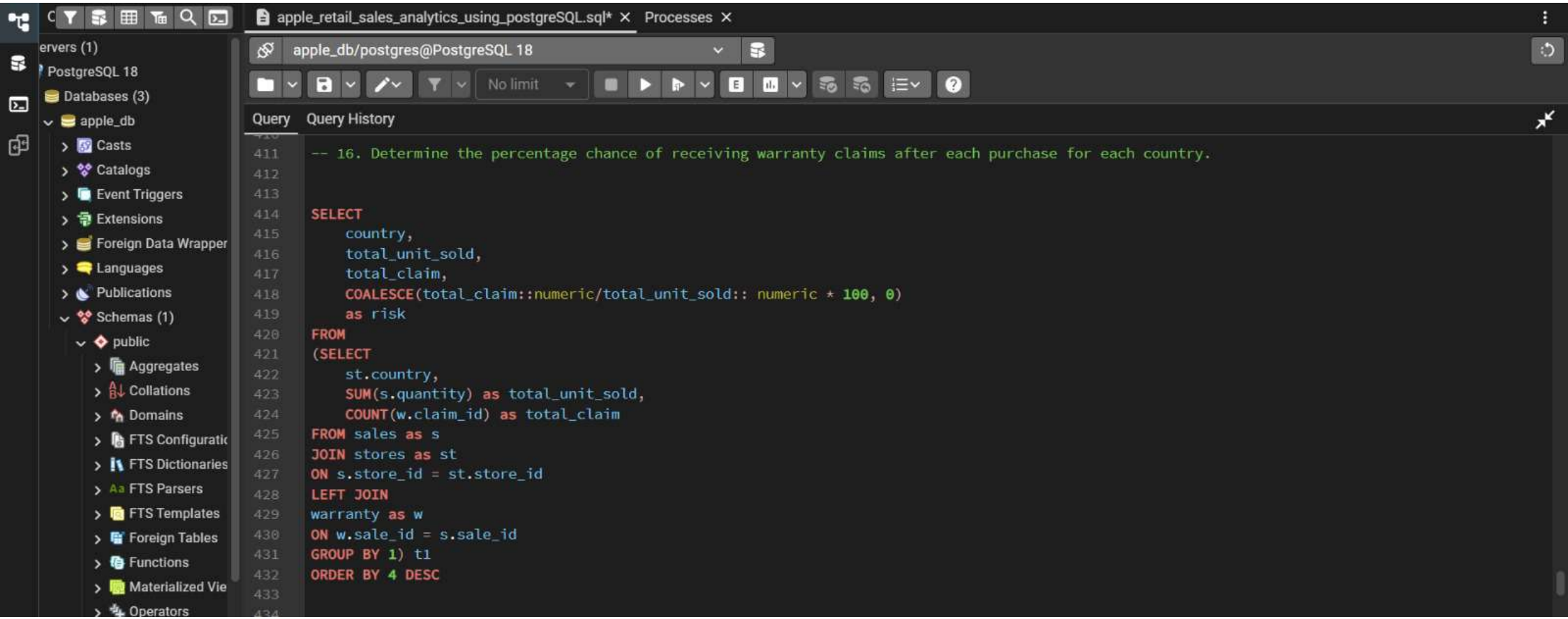

apple_retail_sales_analytics_using_postgreSQL.sql*
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
servers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
Sequences
Tables (5)
category
products
sales
-- 9. How many warranty claims were filed in 2020?
SELECT
COUNT(*) as warranty_claim
FROM warranty
WHERE EXTRACT(YEAR FROM claim_date) = 2024
-- 10. For each store, identify the best-selling day based on highest quantity sold.
SELECT *
FROM(
SELECT
store_id,
TO_CHAR(sale_date, 'Day') as day_name,
SUM(quantity) as total_unit_sold,
RANK() OVER(PARTITION BY store_id ORDER BY SUM(quantity) DESC) as rank
FROM sales
GROUP BY 1, 2
) as t1
WHERE rank = 1
Total rows: 326
Query complete 00:00:00.828
CRLF
Ln 301, Col 1

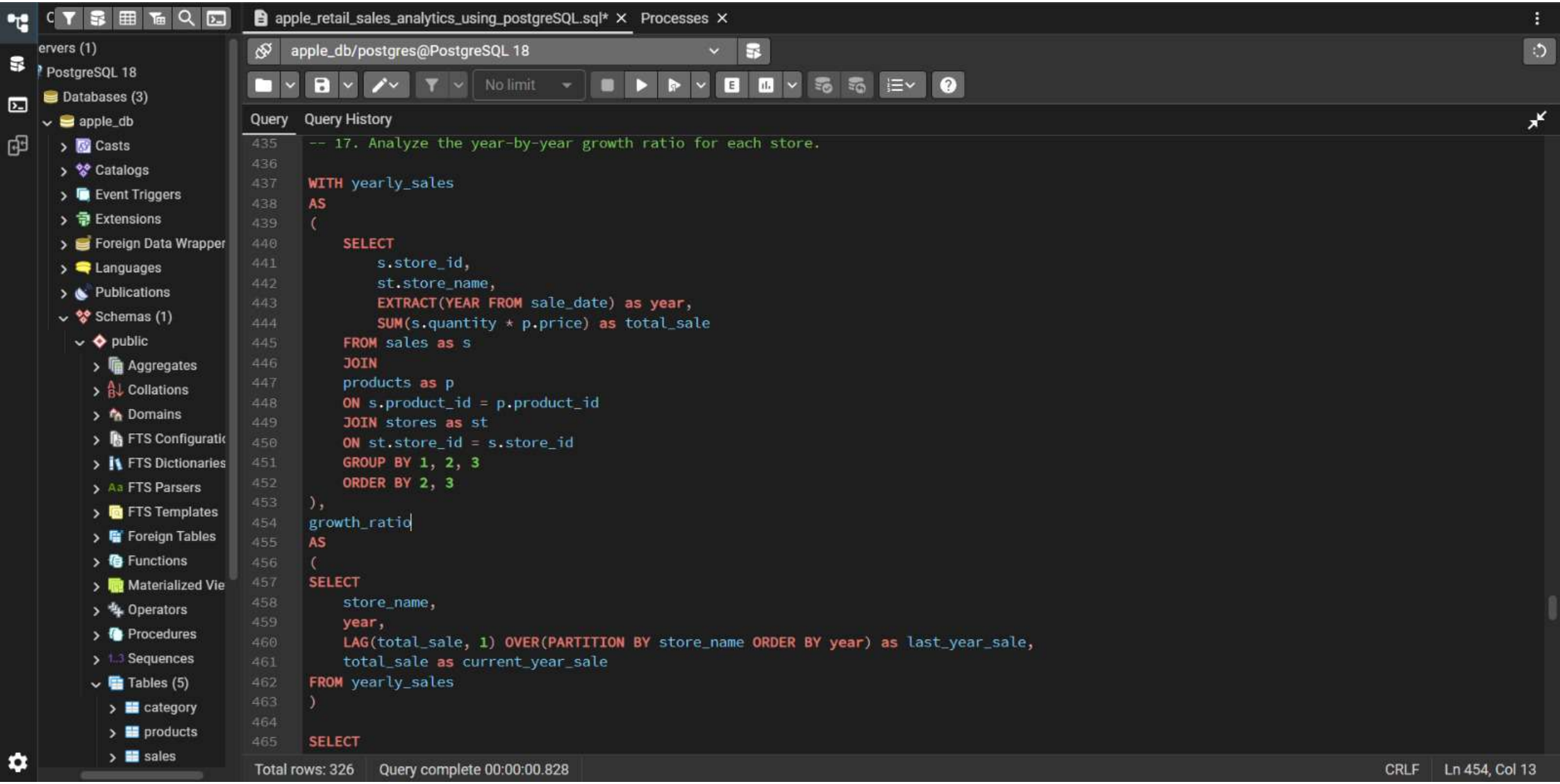

apple_retail_sales_analytics_using_postgreSQL.sql*
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
servers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
303 -- 11. Identify the least selling product in each country for each year based on total units sold.
304
305 WITH product_rank
306 AS
307 (
308 SELECT
309 st.country,
310 p.product_name,
311 sum(s.quantity) as total_qty_sold,
312 RANK() OVER(PARTITION BY st.country ORDER BY SUM(s.quantity)) as rank
313 FROM sales as s
314 JOIN
315 stores as st
316 ON s.store_id = st.store_id
317 JOIN
318 products as p
319 ON s.product_id = p.product_id
320 GROUP BY 1, 2
321 )
322
323 SELECT
324 *
325 FROM product_rank
326 WHERE rank = 1
327
328

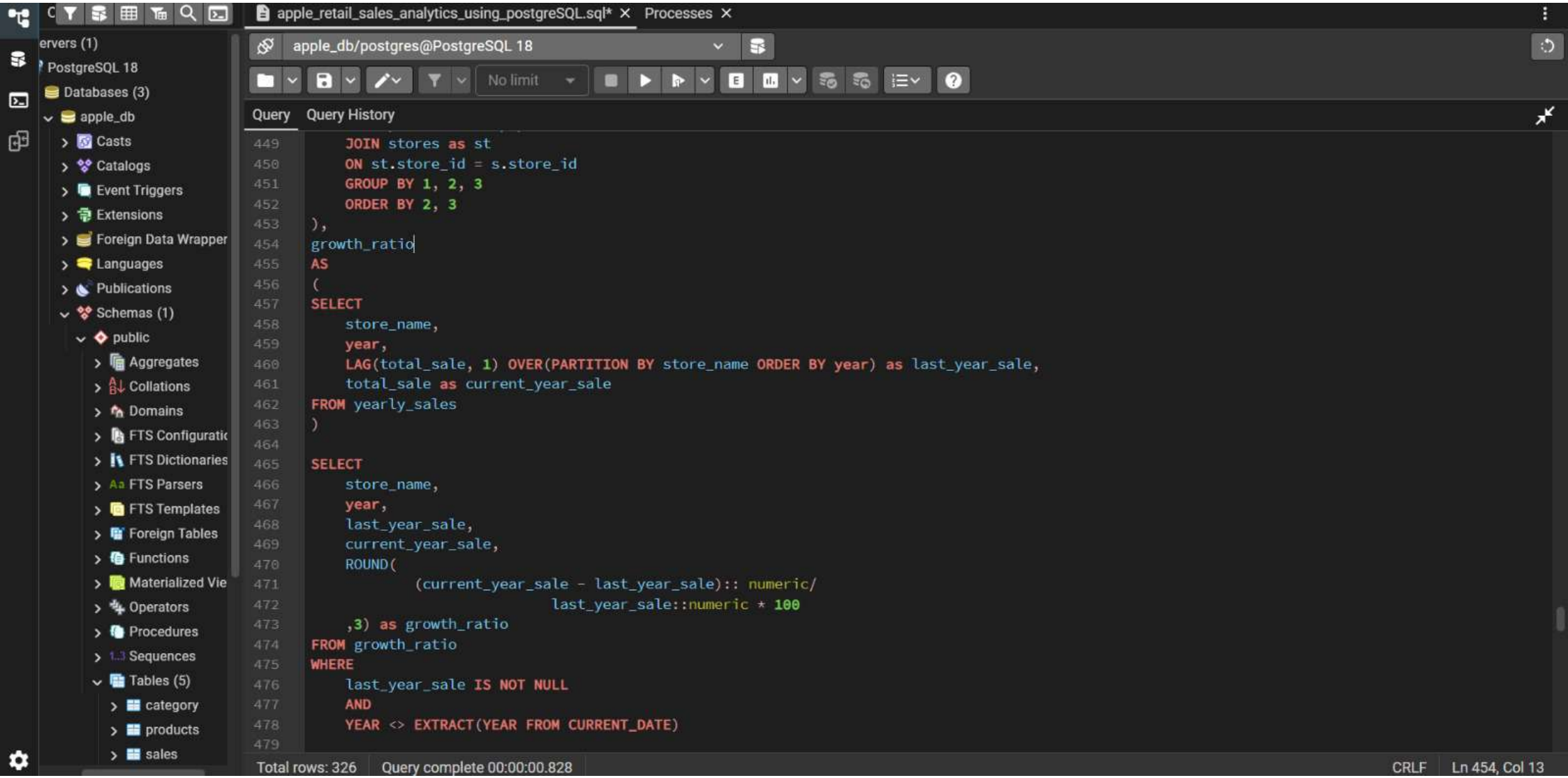

apple_retail_sales_analytics_using_postgreSQL.sql*
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
servers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
-- 12. Calculate how many warranty claims were filed within 180 days of a product sale.
SELECT
w.*,
s.sale_date,
w.claim_date - s.sale_date
FROM warranty as w
LEFT JOIN
sales as s
ON s.sale_id = w.sale_id
WHERE
w.claim_date - s.sale_date <= 180
SELECT
count(*)
FROM warranty as w
LEFT JOIN
sales as s
ON s.sale_id = w.sale_id
WHERE
w.claim_date - s.sale_date <= 180

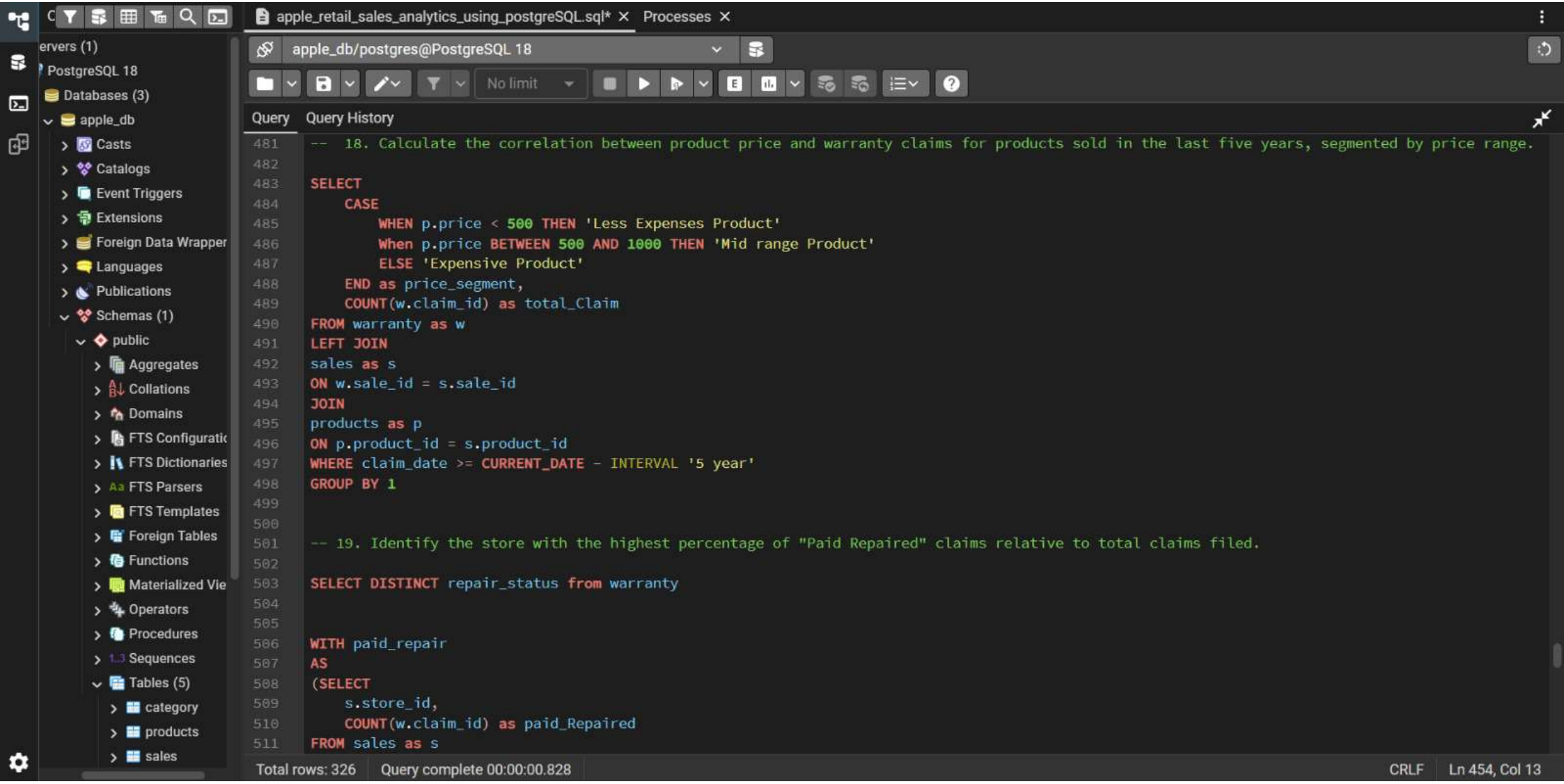

apple_retail_sales_analytics_using_postgreSQL.sql*
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
Servers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
Sequences
Tables (5)
category
products
sales
352 -- 13. Determine how many warranty claims were filed for products launched in the last two years.
353
354 SELECT
355 p.product_name,
356 COUNT(w.claim_id) as no_claim,
357 COUNT(s.sale_id)
358 FROM warranty as w
359 RIGHT JOIN
360 sales as s
361 ON s.sale_id = w.sale_id
362 JOIN products as p
363 ON p.product_id = s.product_id
364 WHERE p.launch_date >= CURRENT_DATE - INTERVAL '3 years'
365 GROUP BY 1
366 HAVING COUNT(w.claim_id) > 0
367
368 -- 14. List the months in the last three years where sales exceeded 5,000 units in the USA.
369
370 SELECT
371 TO_CHAR(sale_date, 'MM-YYYY') as month,
372 SUM(s.quantity) as total_unit_sold
373 FROM sales as s
374 JOIN
375 stores as st
376 ON s.store_id = st.store_id
377 WHERE
378 st.country = 'USA'
379 AND
380 s.sale_date >= CURRENT_DATE - INTERVAL '3 year'
381 GROUP BY 1
382 HAVING SUM(s.quantity)> 5000
Total rows: 326
Query complete 00:00:00.828
CRLF
Ln 301, Col 1

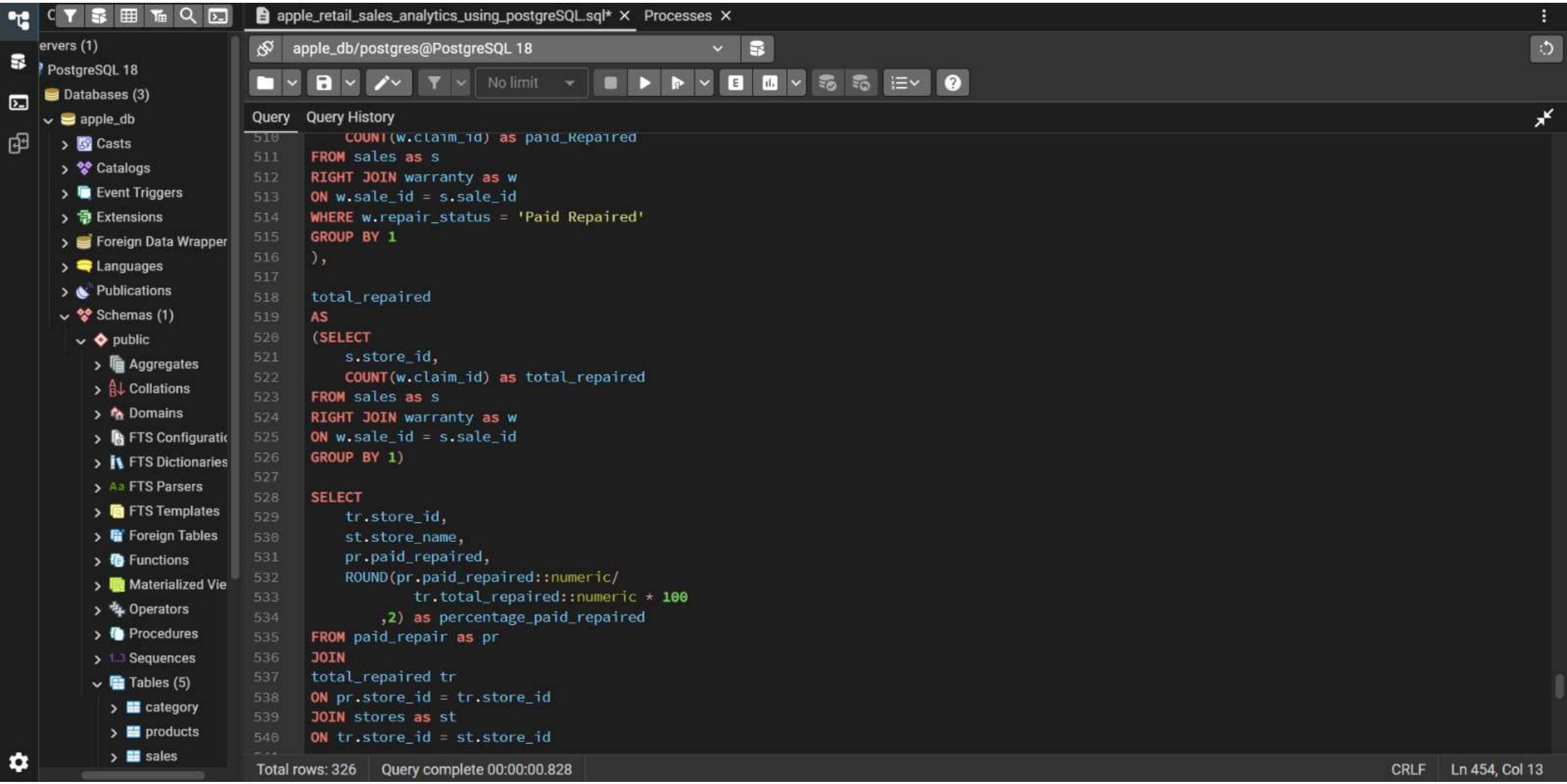

apple_retail_sales_analytics_using_postgreSQL.sql*
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
Servers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
Sequences
Tables (5)
category
products
sales
s.sale_date >= CURRENT_DATE - INTERVAL '3 year'
GROUP BY 1
HAVING SUM(s.quantity)> 5000
-- 15. Identify the product category with the most warranty claims filed in the last two years.
SELECT
c.category_name,
COUNT(w.claim_id) as total_claims
FROM warranty as w
LEFT JOIN
sales as s
ON w.sale_id = s.sale_id
JOIN products as p
ON p.product_id = s.product_id
JOIN
category as c
ON c.category_id = p.category_id
WHERE
w.claim_date >= CURRENT_DATE - INTERVAL '2 year'
GROUP BY 1
Total rows: 326
Query complete 00:00:00.828
CRLF
Ln 409, Col 1

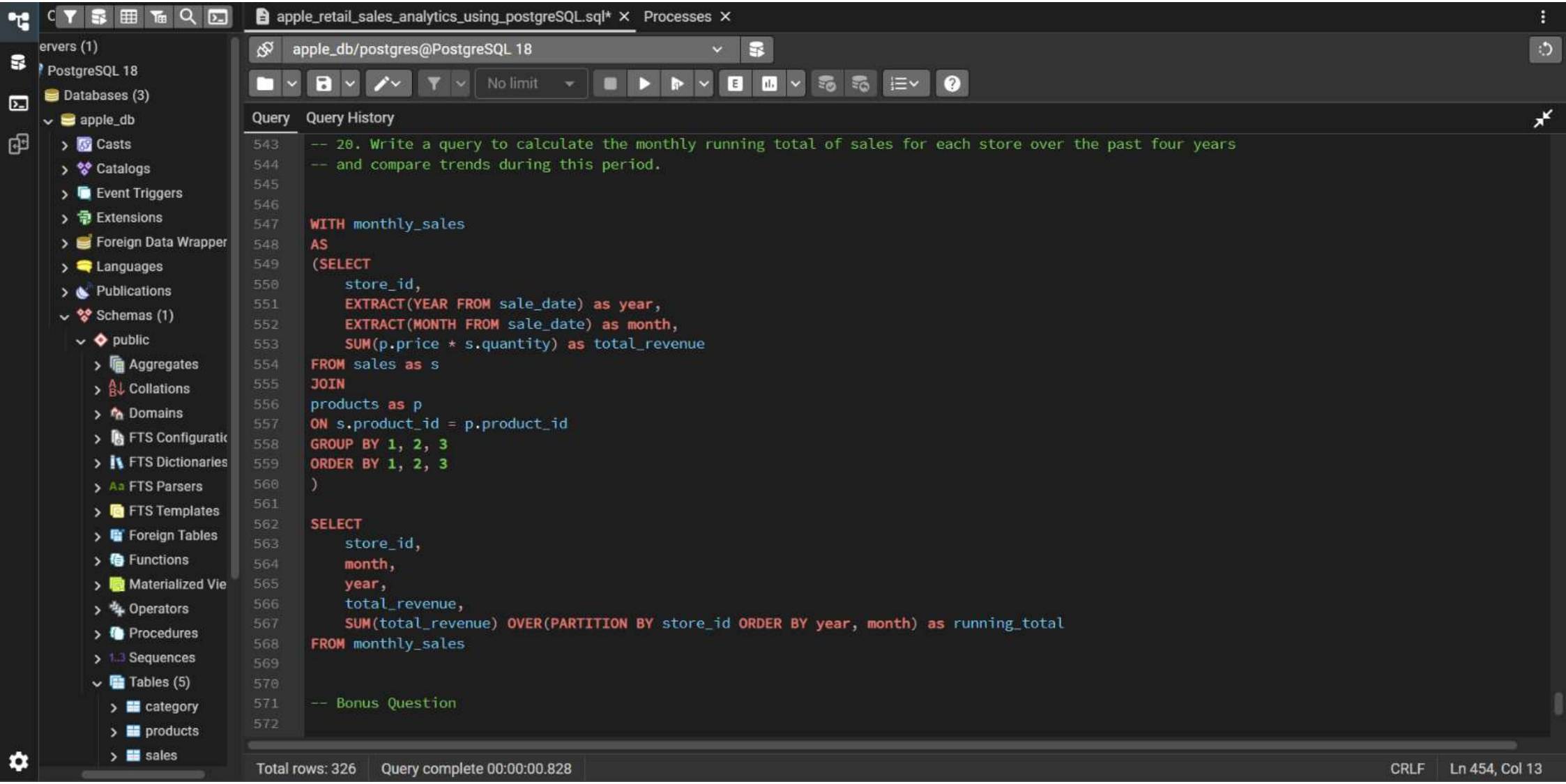

apple_retail_sales_analytics_using_postgreSQL.sql*
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
servers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
411 -- 16. Determine the percentage chance of receiving warranty claims after each purchase for each country.
412
413
414 SELECT
415 country,
416 total_unit_sold,
417 total_claim,
418 COALESCE(total_claim::numeric/total_unit_sold:: numeric * 100, 0)
419 as risk
420 FROM
421 (SELECT
422 st.country,
423 SUM(s.quantity) as total_unit_sold,
424 COUNT(w.claim_id) as total_claim
425 FROM sales as s
426 JOIN stores as st
427 ON s.store_id = st.store_id
428 LEFT JOIN
429 warranty as w
430 ON w.sale_id = s.sale_id
431 GROUP BY 1) t1
432 ORDER BY 4 DESC
433
434

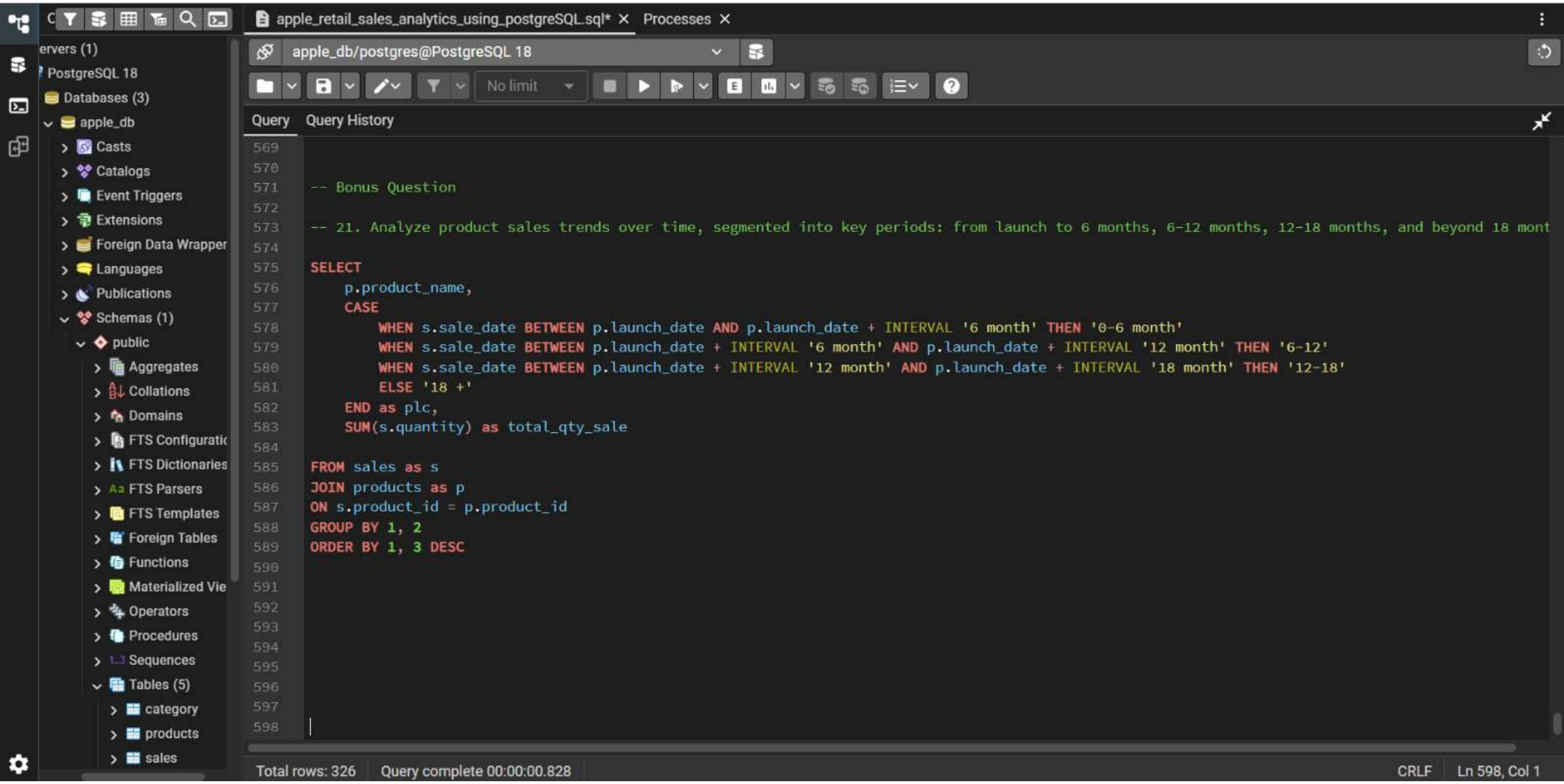

apple_retail_sales_analytics_using_postgreSQL.sql*
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
-- 17. Analyze the year-by-year growth ratio for each store.
WITH yearly_sales
AS
(
SELECT
s.store_id,
st.store_name,
EXTRACT(YEAR FROM sale_date) as year,
SUM(s.quantity * p.price) as total_sale
FROM sales as s
JOIN
products as p
ON s.product_id = p.product_id
JOIN stores as st
ON st.store_id = s.store_id
GROUP BY 1, 2, 3
ORDER BY 2, 3
),
growth_ratio
AS
(
SELECT
store_name,
year,
LAG(total_sale, 1) OVER(PARTITION BY store_name ORDER BY year) as last_year_sale,
total_sale as current_year_sale
FROM yearly_sales
)
SELECT
Total rows: 326
Query complete 00:00:00.828
CRLF
Ln 454, Col 13


apple_retail_sales_analytics_using_postgreSQL.sql*
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
servers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
Sequences
Tables (5)
category
products
sales
449 JOIN stores as st
450 ON st.store_id = s.store_id
451 GROUP BY 1, 2, 3
452 ORDER BY 2, 3
453 ),
454 growth_ratio
455 AS
456 (
457 SELECT
458 store_name,
459 year,
460 LAG(total_sale, 1) OVER(PARTITION BY store_name ORDER BY year) as last_year_sale,
461 total_sale as current_year_sale
462 FROM yearly_sales
463 )
464
465 SELECT
466 store_name,
467 year,
468 last_year_sale,
469 current_year_sale,
470 ROUND(
471 (current_year_sale - last_year_sale):: numeric/
472 last_year_sale::numeric * 100
473 ,3) as growth_ratio
474 FROM growth_ratio
475 WHERE
476 last_year_sale IS NOT NULL
477 AND
478 YEAR <> EXTRACT(YEAR FROM CURRENT_DATE)
479
Total rows: 326
Query complete 00:00:00.828
CRLF
Ln 454, Col 13


apple_retail_sales_analytics_using_postgreSQL.sql*
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
servers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
Sequences
Tables (5)
category
products
sales
481 -- 18. Calculate the correlation between product price and warranty claims for products sold in the last five years, segmented by price range.
482
483 SELECT
484 CASE
485 WHEN p.price < 500 THEN 'Less Expenses Product'
486 When p.price BETWEEN 500 AND 1000 THEN 'Mid range Product'
487 ELSE 'Expensive Product'
488 END as price_segment,
489 COUNT(w.claim_id) as total_Claim
490 FROM warranty as w
491 LEFT JOIN
492 sales as s
493 ON w.sale_id = s.sale_id
494 JOIN
495 products as p
496 ON p.product_id = s.product_id
497 WHERE claim_date >= CURRENT_DATE - INTERVAL '5 year'
498 GROUP BY 1
499
500
501 -- 19. Identify the store with the highest percentage of "Paid Repaired" claims relative to total claims filed.
502
503 SELECT DISTINCT repair_status from warranty
504
505
506 WITH paid_repair
507 AS
508 (SELECT
509 s.store_id,
510 COUNT(w.claim_id) as paid_Repaired
511 FROM sales as s
Total rows: 326
Query complete 00:00:00.828
CRLF
Ln 454, Col 13


apple_retail_sales_analytics_using_postgreSQL.sql*
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
Sequences
Tables (5)
category
products
sales
COUNT(w.claim_id) as paid_Repaired
FROM sales as s
RIGHT JOIN warranty as w
ON w.sale_id = s.sale_id
WHERE w.repair_status = 'Paid Repaired'
GROUP BY 1
),
total_repaired
AS
(SELECT
s.store_id,
COUNT(w.claim_id) as total_repaired
FROM sales as s
RIGHT JOIN warranty as w
ON w.sale_id = s.sale_id
GROUP BY 1)
SELECT
tr.store_id,
st.store_name,
pr.paid_repaired,
ROUND(pr.paid_repaired::numeric/
tr.total_repaired::numeric * 100
,2) as percentage_paid_repaired
FROM paid_repair as pr
JOIN
total_repaired tr
ON pr.store_id = tr.store_id
JOIN stores as st
ON tr.store_id = st.store_id
Total rows: 326
Query complete 00:00:00.828
CRLF
Ln 454, Col 13


apple_retail_sales_analytics_using_postgreSQL.sql*
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
servers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
Sequences
Tables (5)
category
products
sales
543 -- 20. Write a query to calculate the monthly running total of sales for each store over the past four years
544 -- and compare trends during this period.
545
546
547 WITH monthly_sales
548 AS
549 (SELECT
550 store_id,
551 EXTRACT(YEAR FROM sale_date) as year,
552 EXTRACT(MONTH FROM sale_date) as month,
553 SUM(p.price * s.quantity) as total_revenue
554 FROM sales as s
555 JOIN
556 products as p
557 ON s.product_id = p.product_id
558 GROUP BY 1, 2, 3
559 ORDER BY 1, 2, 3
560 )
561
562 SELECT
563 store_id,
564 month,
565 year,
566 total_revenue,
567 SUM(total_revenue) OVER(PARTITION BY store_id ORDER BY year, month) as running_total
568 FROM monthly_sales
569
570
571 -- Bonus Question
572
Total rows: 326
Query complete 00:00:00.828
CRLF
Ln 454, Col 13


apple_retail_sales_analytics_using_postgreSQL.sql*
Processes
apple_db/postgres@PostgreSQL 18
No limit
Query
Query History
servers (1)
PostgreSQL 18
Databases (3)
apple_db
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrapper
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configuratic
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Vie
Operators
Procedures
Sequences
Tables (5)
category
products
sales
569
570
571 -- Bonus Question
572
573 -- 21. Analyze product sales trends over time, segmented into key periods: from launch to 6 months, 6-12 months, 12-18 months, and beyond 18 mont
574
575 SELECT
576 p.product_name,
577 CASE
578 WHEN s.sale_date BETWEEN p.launch_date AND p.launch_date + INTERVAL '6 month' THEN '0-6 month'
579 WHEN s.sale_date BETWEEN p.launch_date + INTERVAL '6 month' AND p.launch_date + INTERVAL '12 month' THEN '6-12'
580 WHEN s.sale_date BETWEEN p.launch_date + INTERVAL '12 month' AND p.launch_date + INTERVAL '18 month' THEN '12-18'
581 ELSE '18 +'
582 END as plc,
583 SUM(s.quantity) as total_qty_sale
584
585 FROM sales as s
586 JOIN products as p
587 ON s.product_id = p.product_id
588 GROUP BY 1, 2
589 ORDER BY 1, 3 DESC
590
591
592
593
594
595
596
597
598
Total rows: 326
Query complete 00:00:00.828
CRLF
Ln 598, Col 1